LAPORAN PRAKTIKUM

REKAYASA SISTEM INFORMASI

Git dan *Version Control*

Link GitHub

Disusun Oleh:

Trisha Garnis Wahningyun	(L0224012)
Alvian Damar Budhi Hernowo	(L0224014)
Fathul Fajar Nur Ikhsan	(L0224018)
Kunto Rossindu Hidayattullah	(L0224020)
Zaki Elias Al Haqqanikudus	(L0224039)

PROGRAM STUDI S1 SAINS DATA

FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA

UNIVERSITAS SEBELAS MARET

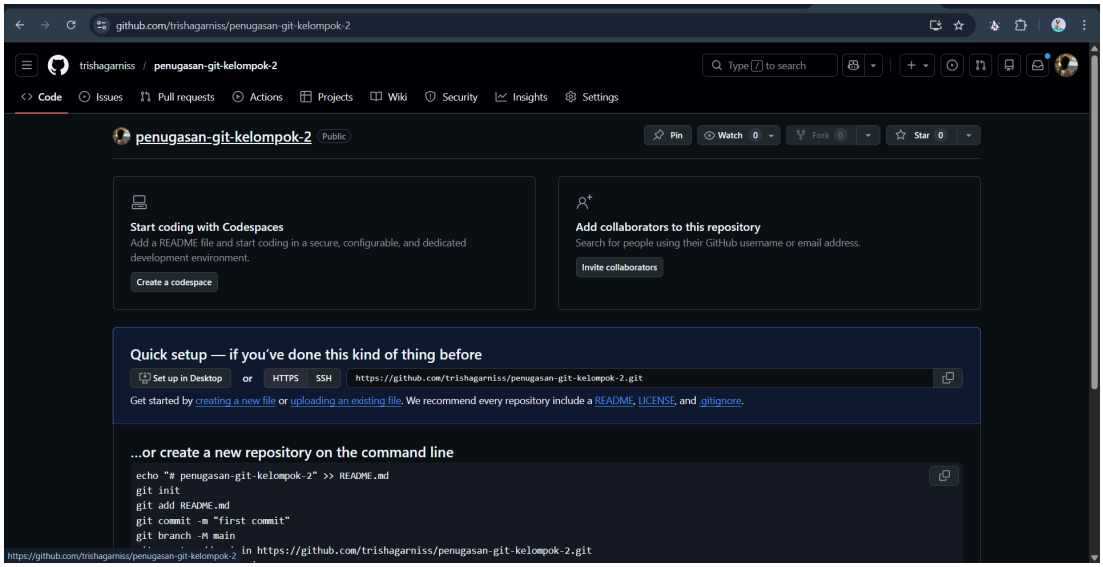
2026

DOKUMENTASI PENGERJAAN

Langkah 1: Inisialisasi Repository dan Pembuatan Branch

A. Membuat Repository di Website GitHub

1. Buka akun GitHub melalui peramban web (browser).
2. Klik tombol **New** (atau logo + di sudut kanan atas, kemudian pilih **New repository**).
3. Pada kolom **Repository name,** ketik nama repository: `penugasan-git-kelompok-2`
4. Pilih opsi **Public** pada pengaturan visibilitas repositori.
5. Biarkan kotak centang "Add a README file" dalam keadaan kosong (tidak dicentang) agar inisialisasi dapat dilakukan secara lokal.
6. Klik tombol **Create repository**.
7. Salin tautan (URL) repository yang telah berhasil dibuat untuk digunakan pada langkah selanjutnya.

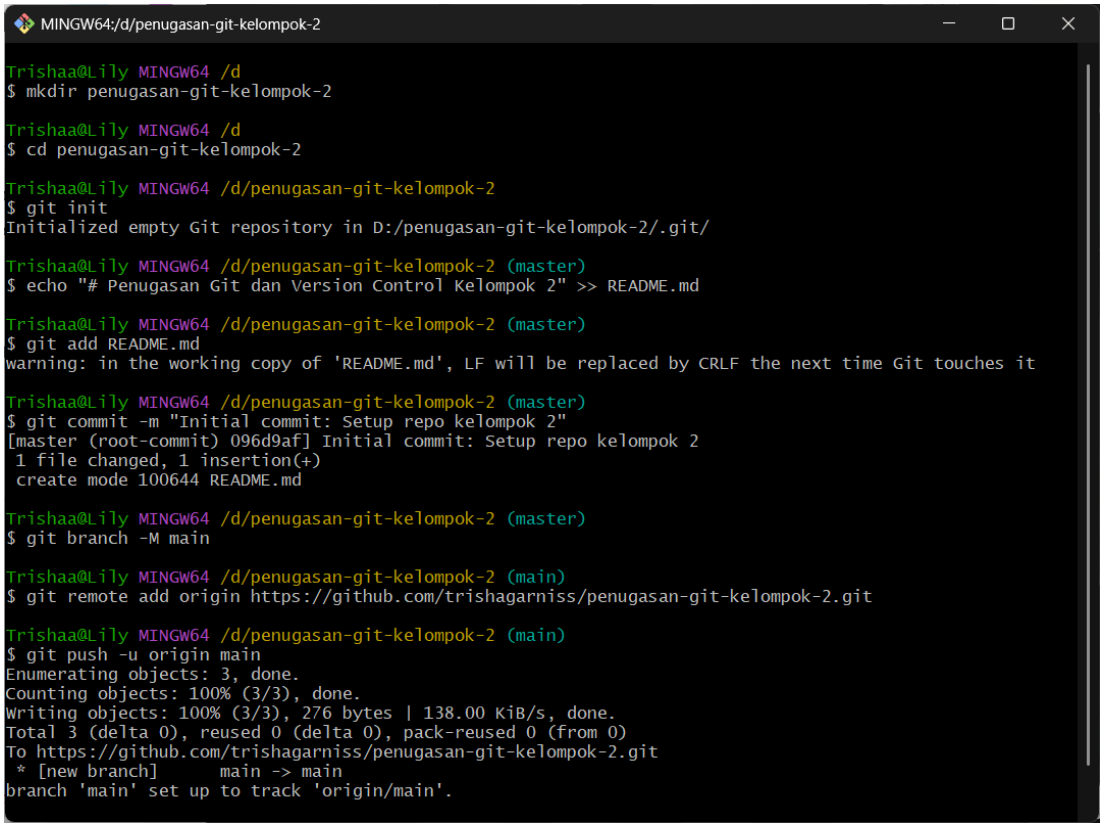


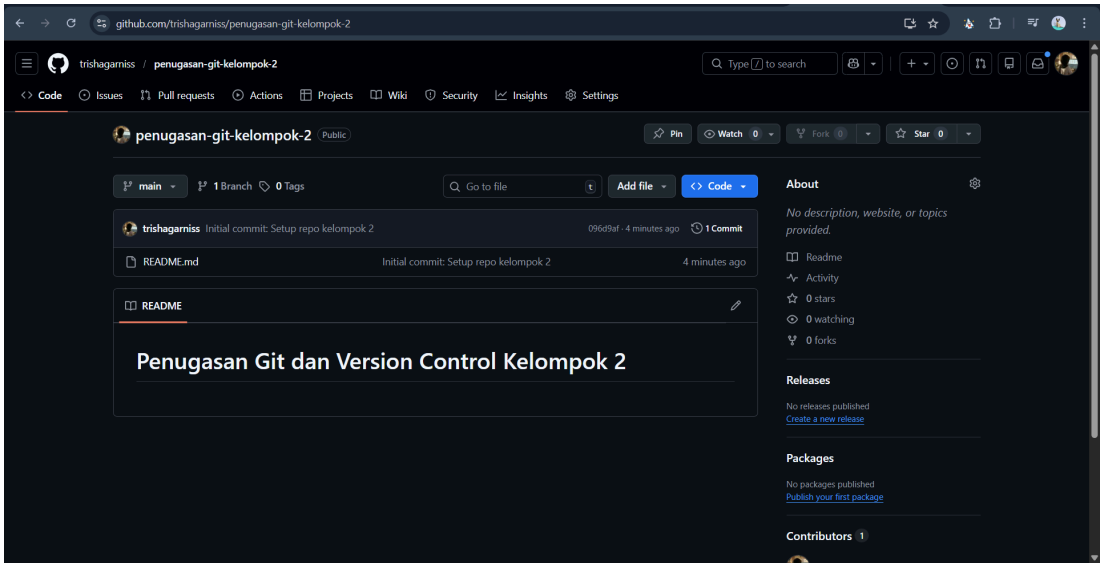
B. Menginisialisasi Repository secara Lokal melalui Terminal / Git Bash

1. Buka Terminal atau Git Bash pada perangkat komputer.
2. Jalankan serangkaian perintah terminal berikut secara berurutan untuk membuat direktori, menginisialisasi Git, dan menghubungkan dengan repository GitHub:

```
mkdir penugasan-git-kelompok-2
cd penugasan-git-kelompok-2
git init
echo "# Penugasan Git dan Version Control Kelompok 2" > README.md
```

```
git add README.md
git commit -m "Initial commit: Setup repo kelompok 2"
git branch -M main
git remote add origin [URL_REPOSITORY]
git push -u origin main
```

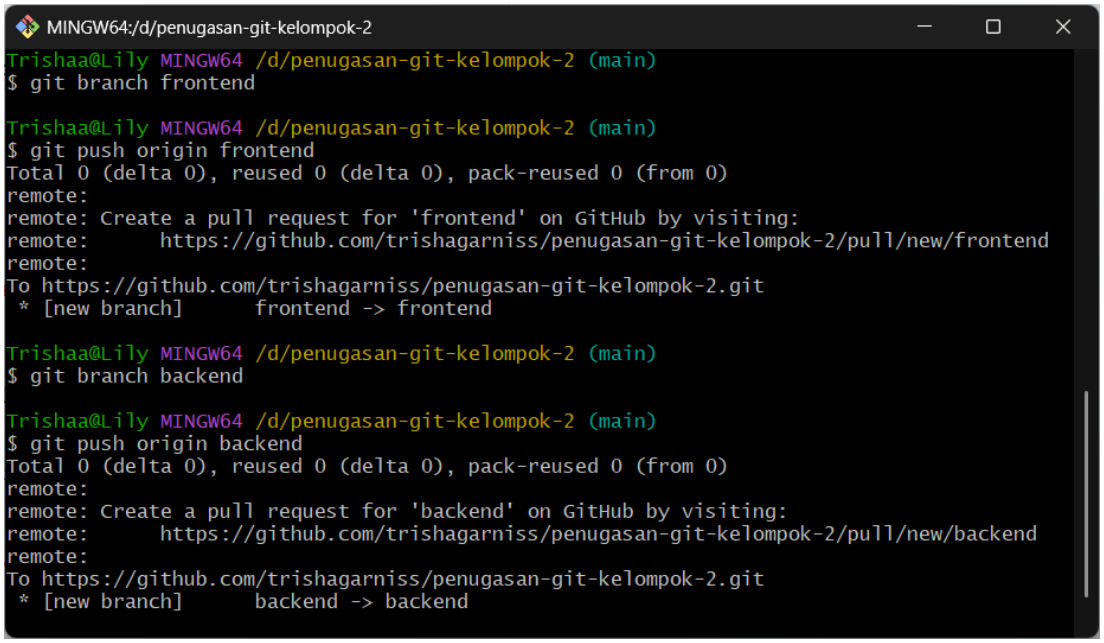




C. Bikin Branch Frontend dan Backend

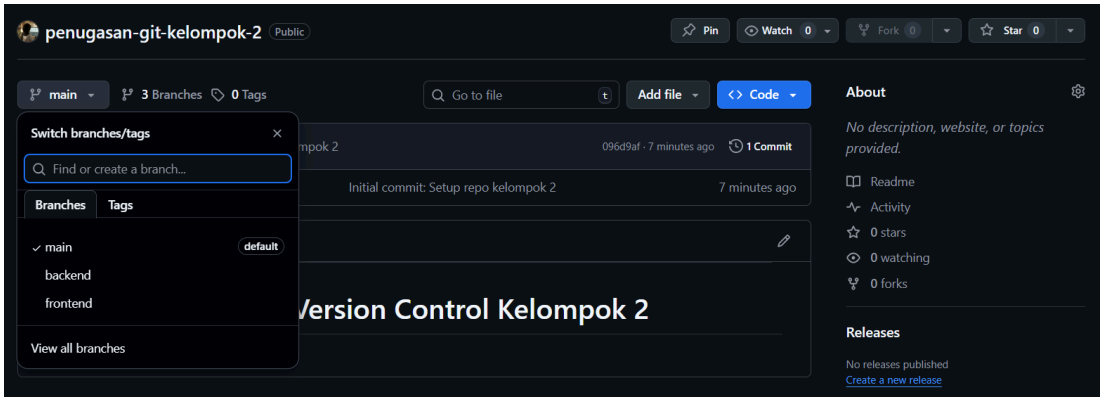
Setelah *commit* pertama berhasil diunggah ke repositori utama *(main),* jalankan perintah berikut pada terminal yang sama untuk membuat dan mengunggah *branch* baru:

```
git branch frontend
git push origin frontend
git branch backend
git push origin backend
```



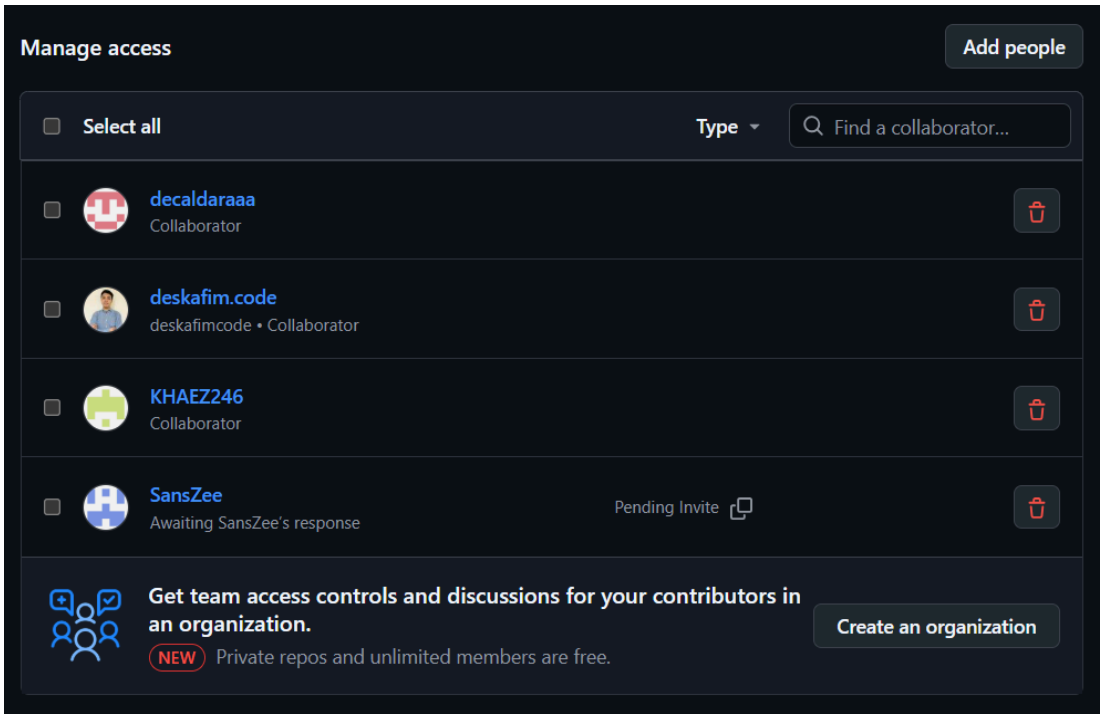
D. Memverifikasi Hasil Inisialisasi

1. Buka kembali halaman repository GitHub pada peramban web dan muat ulang *(refresh)* halaman tersebut.
2. Pastikan opsi *branch* frontend dan backend telah tersedia pada menu *dropdown branch* di sudut kiri atas.



Langkah 2: Menambahkan Anggota Kelompok sebagai Kolaborator

1. Buka halaman repository penugasan-git-kelompok-2 melalui peramban web (*browser*) di GitHub.
2. Klik *tab* **Settings** yang berada di deretan menu bagian atas repository.
3. Pada panel navigasi di sebelah kiri, pilih menu **Collaborators**.
4. Klik tombol bertuliskan **Add People**.
5. Masukkan *username* GitHub dari seluruh anggota kelompok satu per satu ke dalam kolom pencarian.
6. Pilih akun yang tepat, lalu klik tombol **Add to repository**.
7. Instruksikan seluruh anggota kelompok untuk memeriksa kotak masuk notifikasi di akun GitHub masing-masing, kemudian mengklik **Accept Invitation** agar mereka memiliki izin.



Langkah 3: Menginisialisasi Folder Backend (Poetry dan Python)

1. Buka aplikasi Terminal atau Git Bash pada perangkat komputer.
2. Lakukan kloning repositori GitHub yang telah dibuat sebelumnya dengan mengetikkan perintah:

```
git clone https://github.com/trishagarniss/penugasan-git-kelompok-2.git
```

3. Masuk ke dalam direktori repository tersebut dengan perintah:

   `cd penugasan-git-kelompok-2`

4. Berpindah ke *branch* backend dengan mengetikkan:

   `git checkout backend`

5. Buat dan inisialisasi folder backend menggunakan Poetry dengan menjalankan perintah:

   `poetry new -i backend`

6. Masuk ke dalam folder backend yang baru saja dibuat, kemudian instal *environment* Python:

   `cd backend`

   `poetry install`

7. Kembali ke direktori utama repositori untuk menyiapkan penyimpanan perubahan:

   `cd ..`

8. Tambahkan folder backend tersebut ke dalam Git:

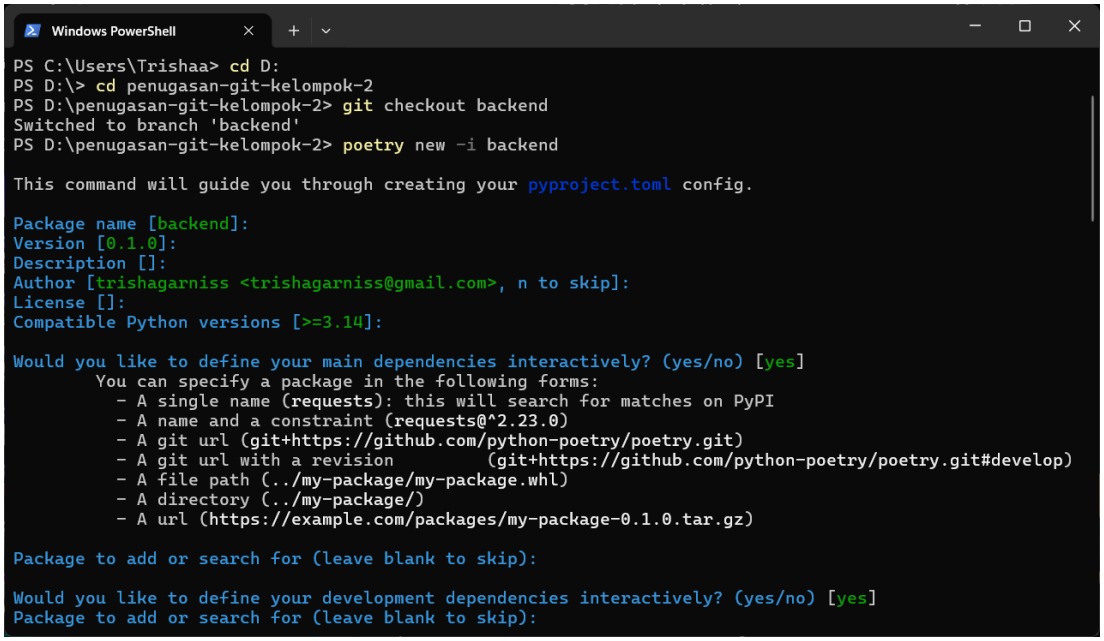
   `git add backend`

9. Simpan perubahan dengan membuat *commit*:

   `git commit -m "feat: inisialisasi folder backend dengan Poetry dan Python"`

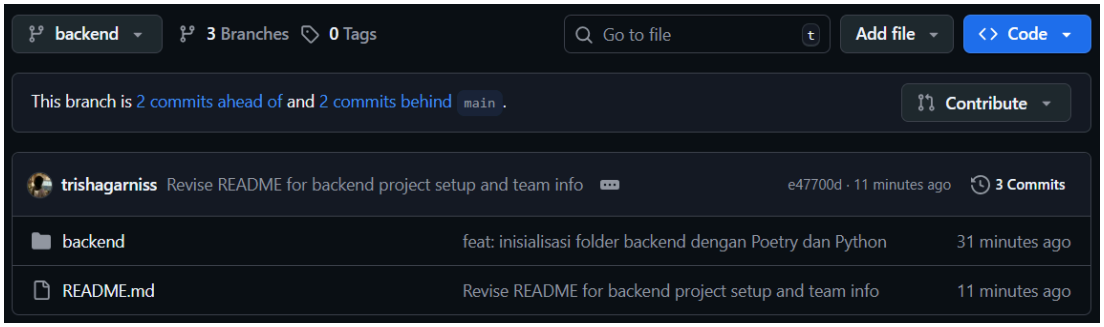
10. Unggah (*push*) seluruh perubahan tersebut ke repositori GitHub pada *branch* backend:

    `git push origin backend`



```
Generated file

[project]
name = "backend"
version = "0.1.0"
description = ""
authors = [
    {name = "trishagarniss",email = "trishagarniss@gmail.com"}
]
readme = "README.md"
requires-python = ">=3.14"
dependencies = [
]

[tool.poetry]
packages = [{include = "backend", from = "src"}]

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"


Do you confirm generation? (yes/no) [yes]
Created package backend in backend
PS D:\penugasan-git-kelompok-2> cd backend
PS D:\penugasan-git-kelompok-2\backend> poetry install
Creating virtualenv backend-rtiihX4T-py3.14 in C:\Users\Trishaa\AppData\Local\pypoetry\Cache\virtualenv
s
Updating dependencies
Resolving dependencies... (0.2s)

Writing lock file

Installing the current project: backend (0.1.0)
```

```
PS D:\penugasan-git-kelompok-2\backend> cd ..
PS D:\penugasan-git-kelompok-2> git add backend
PS D:\penugasan-git-kelompok-2> git commit -m "feat: inisialisasi folder backend dengan Poetry dan Python"
[backend c1de289] feat: inisialisasi folder backend dengan Poetry dan Python
 5 files changed, 25 insertions(+)
 create mode 100644 backend/README.md
 create mode 100644 backend/poetry.lock
 create mode 100644 backend/pyproject.toml
 create mode 100644 backend/src/backend/__init__.py
 create mode 100644 backend/tests/__init__.py
PS D:\penugasan-git-kelompok-2> git push origin backend
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 1024 bytes | 21.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/trishagarniss/penugasan-git-kelompok-2.git
   096d9af..c1de289  backend -> backend
PS D:\penugasan-git-kelompok-2> git log --oneline
```



Langkah 4: Menginisialisasi Folder Frontend (React + NextJS)

1. Buka aplikasi Terminal atau Git Bash pada perangkat komputer.
2. Lakukan kloning repositori GitHub dengan mengetikkan perintah berikut:

```
git clone https://github.com/trishagarniss/penugasan-git-kelompok-2.git
```

3. Masuk ke dalam direktori repository tersebut:

```
cd penugasan-git-kelompok-2
```

4. Berpindah ke *branch* frontend untuk memastikan perubahan tidak mengganggu *branch* utama:

```
git checkout frontend
```

5. Buat dan inisialisasi folder *frontend* menggunakan kerangka kerja React dan NextJS dengan menjalankan perintah berikut:

```
npx create-next-app@latest frontend
```

6. Setelah proses instalasi selesai, tambahkan folder *frontend* tersebut ke dalam Git:

```
git add frontend
```

7. Simpan perubahan dengan membuat *commit*:

```
git commit -m "feat: inisialisasi folder frontend dengan React dan NextJS"
```

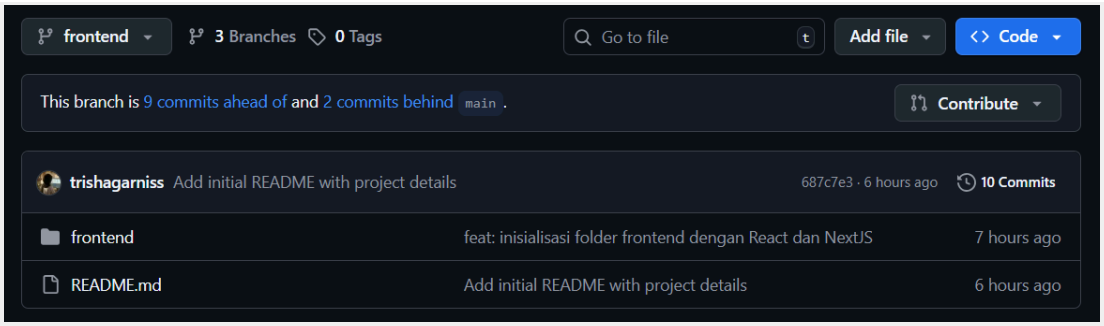
8. Unggah (*push*) seluruh perubahan tersebut ke repository GitHub pada *branch* frontend:

```
git push origin frontend
```

```
PS D:\SEMESTER 4\RSI> git clone https://github.com/trishagarniss/penugasan-git-kelompok-2.git
Cloning into 'penugasan-git-kelompok-2'...
remote: Enumerating objects: 58, done.
remote: Counting objects: 100% (58/58), done.
remote: Compressing objects: 100% (42/42), done.
remote: Total 58 (delta 6), reused 49 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (58/58), 157.25 KiB | 1.73 MiB/s, done.
Resolving deltas: 100% (6/6), done.
PS D:\SEMESTER 4\RSI> cd penugasan-git-kelompok-2
PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> git checkout frontend
branch 'frontend' set up to track 'origin/frontend'.
Switched to a new branch 'frontend'
PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> npx create-next-app@latest frontend
√ Would you like to use the recommended Next.js defaults? » Yes, use recommended defaults
Creating a new Next.js app in D:\SEMESTER 4\RSI\penugasan-git-kelompok-2\frontend.

Using npm.

Initializing project with template: app-tw


Installing dependencies:
- next
- react
- react-dom

Installing devDependencies:
- @tailwindcss/postcss
- @types/node
- @types/react
- @types/react-dom
- eslint
- eslint-config-next
- tailwindcss
- typescript
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                  powershell - penu
PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> npx create-next-app@latest frontend

added 359 packages, and audited 360 packages in 1m

143 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

Generating route types...
✓ Types generated successfully

Success! Created frontend at D:\SEMESTER 4\RSI\penugasan-git-kelompok-2\frontend

PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> git add frontend
warning: in the working copy of 'frontend/.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/app/globals.css', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/app/layout.tsx', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/app/page.tsx', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/eslint.config.mjs', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/next.config.ts', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/package.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/postcss.config.mjs', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'frontend/tsconfig.json', LF will be replaced by CRLF the next time Git touches it
PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> git commit -m "feat: inisialisasi folder frontend dengan React dan NextJS"
[frontend a08a868] feat: inisialisasi folder frontend dengan React dan NextJS
 17 files changed, 6892 insertions(+)
 create mode 100644 frontend/.gitignore
 create mode 100644 frontend/README.md
 create mode 100644 frontend/app/favicon.ico
 create mode 100644 frontend/app/globals.css
 create mode 100644 frontend/app/layout.tsx
 create mode 100644 frontend/app/page.tsx
 create mode 100644 frontend/eslint.config.mjs
 create mode 100644 frontend/next.config.ts
 create mode 100644 frontend/package-lock.json
 create mode 100644 frontend/package.json
 create mode 100644 frontend/postcss.config.mjs
 create mode 100644 frontend/public/file.svg
frontend   ⊗ 0 ⚠ 0
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                  powershell - pen

PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> git commit -m "feat: inisialisasi folder frontend dengan React dan NextJS"
 create mode 100644 frontend/public/globe.svg
 create mode 100644 frontend/public/next.svg
 create mode 100644 frontend/public/vercel.svg
 create mode 100644 frontend/public/window.svg
 create mode 100644 frontend/tsconfig.json
PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2> git push origin frontend
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 16 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (22/22), 63.34 KiB | 4.22 MiB/s, done.
Total 22 (delta 0), reused 15 (delta 0), pack-reused 0 (from 0)
To https://github.com/trishagarniss/penugasan-git-kelompok-2.git
   47e7ae2..a08a868  frontend -> frontend
PS D:\SEMESTER 4\RSI\penugasan-git-kelompok-2>
```



Langkah 5: Membuat Perubahan pada Folder Backend (GET, POST, PUT, DELETE)

A. Implementasi dan Pengujian Metode GET dan POST

1. Buka aplikasi Terminal atau Git Bash pada perangkat komputer.
2. Pastikan posisi direktori kerja saat ini berada di dalam *branch* backend dengan menjalankan perintah berikut:

   `git checkout backend`
3. Unduh pembaruan terbaru dari repository (*pull*) untuk menyinkronkan data dan mendapatkan folder hasil inisialisasi yang telah diunggah sebelumnya:

   `git pull origin backend`
4. Buka folder *backend* menggunakan *text editor* (VS Code), kemudian buat sebuah berkas baru (misalnya `main.py`) dan tuliskan kode program dasar untuk menangani *request* dengan metode GET dan POST.

```python
@app.get("/", tags=["General"])
def read_root():
    return {
        "message": "Selamat datang di API Kelompok 2!",
        "petunjuk": "Tambahkan /docs di akhir URL untuk melihat dokumentasi interaktif."
    }

# GET: untuk melihat isi list items
@app.get("/items")
def get_items():
    return items

# POST: untuk menambahkan data ke list items
@app.post("/items")
def create_item(item: Item):
    if item.id in items : # Cek ID sudah ada atau belum
        return {"message": "ID sudah ada, Silahkan pilih ID yang lain", "data": item}
    items[item.id] = item.model_dump() # Ubah class ke dictionary lalu Masukin ke dictionary Items
    return {"message": "Data masuk!", "data": item}
```

5. Setelah kode selesai ditulis dan disimpan, kembali ke terminal untuk mendaftarkan perubahan tersebut ke dalam Git:

   `git add .`
6. Simpan perubahan dengan membuat *commit*:

   `git commit -m "feat: menambahkan endpoint untuk metode GET dan POST pada backend"`
7. Unggah (*push*) kode yang telah diperbarui ke repository GitHub pada *branch* backend:

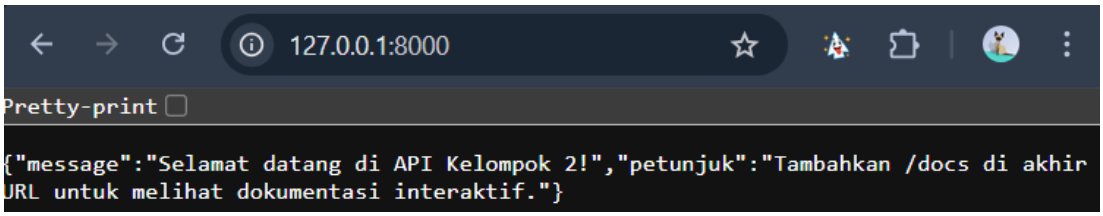
   `git push origin backend`

```
(backend-py3.14) PS D:\penugasan-git-kelompok-2> git add .
(backend-py3.14) PS D:\penugasan-git-kelompok-2> git commit -m "feat: menambahkan endpoint untuk metode GET dan POST pada backend"
[backend 555913e] feat: menambahkan endpoint untuk metode GET dan POST pada backend
 1 file changed, 12 insertions(+), 2 deletions(-)
(backend-py3.14) PS D:\penugasan-git-kelompok-2> git push origin backend
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 616 bytes | 205.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/trishagarniss/penugasan-git-kelompok-2.git
   dc4ae33..555913e  backend -> backend
(backend-py3.14) PS D:\penugasan-git-kelompok-2>
```

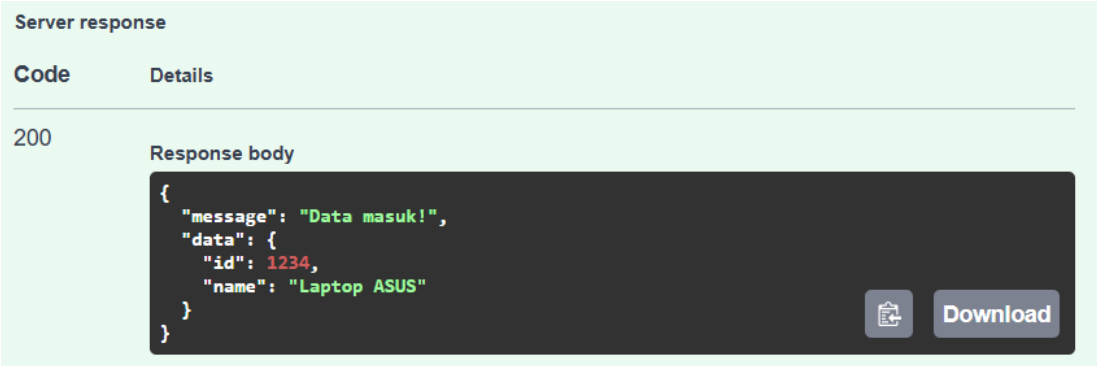
8. Setelah kode berhasil diunggah, lakukan pengujian API secara lokal dengan menjalankan server FastAPI melalui terminal menggunakan perintah berikut:

```
poetry run uvicorn main:app --reload
```

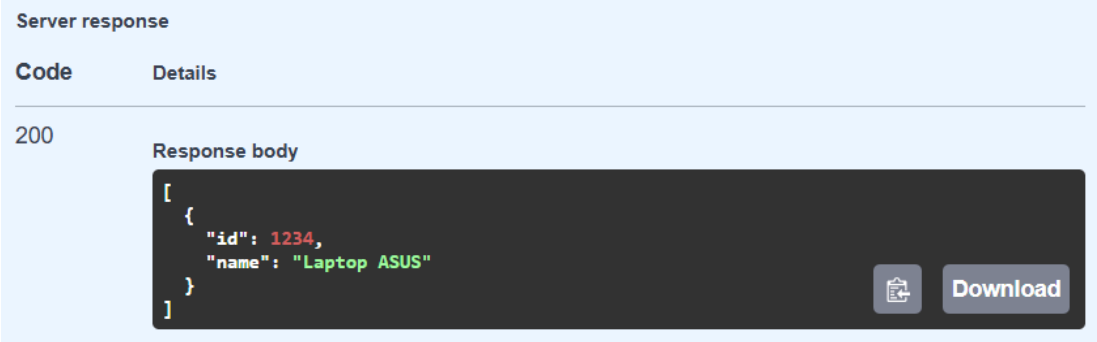
```
(backend-py3.14) PS D:\penugasan-git-kelompok-2\backend> poetry run uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['D:\\penugasan-git-kelompok-2\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [14296] using StatReload
INFO:     Started server process [27020]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



9. Buka *web browser* dan akses alamat http://127.0.0.1:8000/docs untuk membuka antarmuka Swagger UI interaktif yang dihasilkan secara otomatis oleh FastAPI.
10. Lakukan pengujian untuk metode **POST** dengan membuka menu *endpoint* /items, klik tombol "Try it out", masukkan data JSON yang diinginkan pada kolom *Request Body,* dan klik "Execute". Pastikan bagian *Server response* menampilkan pesan sukses.



11. Lakukan pengujian untuk metode **GET** dengan membuka menu *endpoint* /items, klik tombol "Try it out", lalu klik "Execute" untuk memverifikasi bahwa data yang baru saja dikirim telah berhasil tersimpan dan ditampilkan dengan benar oleh server.



B. Implementasi dan Pengujian Metode PUT dan DELETE

1. Buka Terminal atau Git Bash pada perangkat komputer, lalu pastikan berada di *branch* yang tepat:

   `git checkout backend`

2. Unduh pembaruan terbaru dari repository (*pull*) untuk mendapatkan berkas `main.py` yang sebelumnya telah memuat metode GET dan POST:

   `git pull origin backend`

3. Buka berkas `main.py` tersebut pada *text editor*, lalu modifikasi kodenya dengan menambahkan fungsi lanjutan untuk menangani *request* dengan metode PUT dan DELETE.

```
@app.put("/items")
def put_item(item: Item):
    if item.id in items :
        item_lama = items[item.id]
        items[item.id] = item
        return {"message": "Data diubah!", "data lama": item_lama, "data baru" : item}
    else :
        return {"message": "Tidak Ada Data Tersebut Didalam List", "data": item}

@app.delete("/items")
def delete_item(item: Item):
    if item.id in items :
        del items[item.id]
        return {"message": "Data dihapus!", "data": item}
    else :
        return {"message": "Tidak Ada Data Tersebut Didalam List", "data": item}
```

4. Setelah dimodifikasi kode selesai dan disimpan, daftarkan perubahan tersebut ke dalam Git melalui terminal:

   `git add .`

5. Simpan perubahan tersebut dengan membuat *commit:*

   `git commit -m "feat: menambahkan endpoint untuk metode PUT dan DELETE pada backend"`

6. Unggah (*push*) seluruh pembaruan ke repository GitHub:

   `git push origin backend`

7. Setelah kode berhasil diunggah, lakukan pengujian API secara lokal dengan menjalankan server FastAPI melalui terminal menggunakan perintah berikut:

   `poetry run uvicorn main:app --reload`

```
PS C:\Prak_RSI\penugasan-git-kelompok-2\backend> poetry run uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Prak_RSI\\penugasan-git-kelompok-2\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [7860] using StatReload
INFO:     Started server process [15748]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

8. Buka *web browser* dan akses alamat http://127.0.0.1:8000/docs untuk membuka antarmuka Swagger UI interaktif yang dihasilkan secara otomatis oleh FastAPI.
9. Lakukan pengujian untuk metode **PUT** dengan membuka menu *endpoint* /items, klik tombol "Try it out", masukkan data JSON yang diinginkan pada kolom *Request Body,* dan klik "Execute". Pastikan bagian *Server response* menampilkan pesan sukses.

Server response

Code	Details
200	**Response body**

```
{
  "message": "Tidak Ada Data Tersebut Didalam List",
  "data": {
    "id": 1234,
    "name": "PC ASUS"
  }
}
```

Server response

Code	Details
200	Response body

```
{
  "message": "Data diubah!",
  "data lama": {
    "id": 1234,
    "name": "PC ASUS"
  },
  "data baru": {
    "id": 1234,
    "name": "Laptop ASUS"
  }
}
```

10. Lakukan pengujian untuk metode **DELETE** dengan membuka menu endpoint /items/{id}, klik tombol "Try it out". Masukkan ID dari data yang ingin dihapus pada kolom parameter yang tersedia (tanpa menggunakan *Request body*), kemudian klik tombol "Execute". Pastikan bagian Server response menampilkan pesan sukses (kode 200) sebagai indikasi bahwa data telah berhasil dihapus dari sistem.

Server response

Code	Details
200	Response body

```
{
  "message": "Tidak Ada Data Tersebut Didalam List",
  "data": {
    "id": 1234,
    "name": "Laptop ACER"
  }
}
```

Server response

Code	Details
200	Response body

```
{
  "message": "Data dihapus!",
  "data": {
    "id": 1234,
    "name": "Laptop ASUS"
  }
}
```

Langkah 6: Membuat Perubahan pada Folder Frontend (Homepage)

A. Persiapan Lingkungan Local (Local Setup)

1. Buka aplikasi Terminal atau Git Bash pada perangkat komputer.
2. Pastikan posisi direktori kerja saat ini berada di dalam *branch* frontend dengan menjalankan perintah berikut:

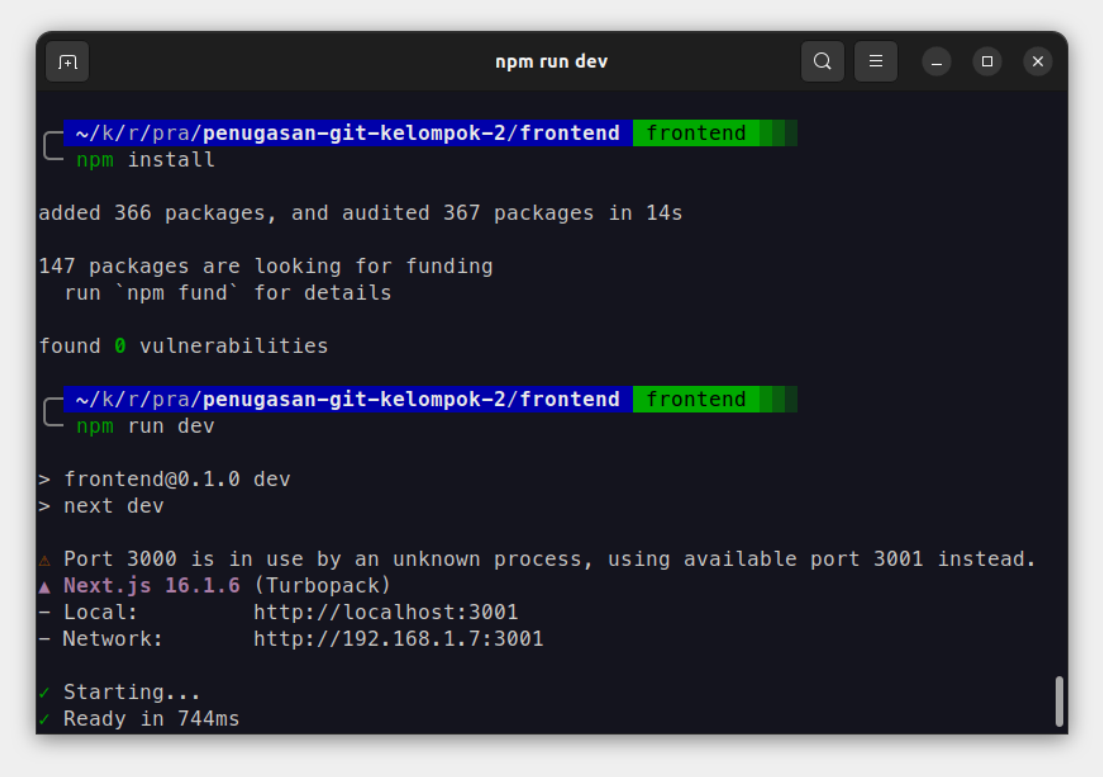
   `git checkout frontend`
3. Unduh pembaruan terbaru dari repositori (*pull*) untuk mendapatkan kerangka folder React dan NextJS yang sebelumnya telah diinisialisasi:

   `git pull origin frontend`
4. Masuk ke dalam direktori folder frontend:

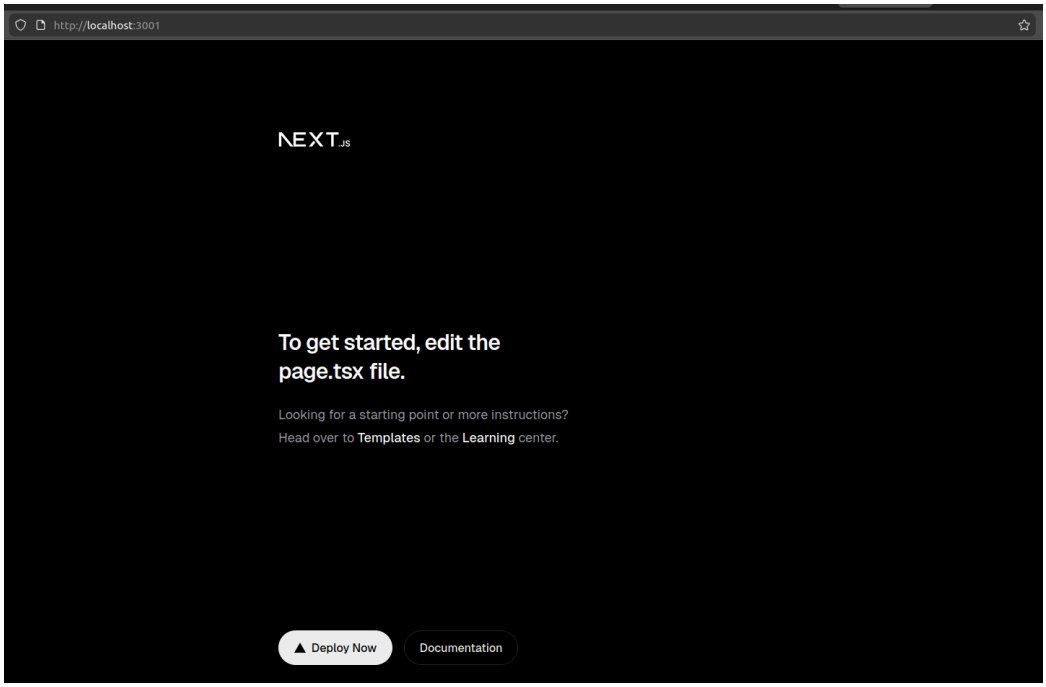
   `cd frontend`
5. Lakukan instalasi semua *dependencies* atau *library* yang dibutuhkan oleh React/Next.js dengan menjalankan perintah:

   `npm install`
6. Jalankan *server* pengembangan lokal (Local Development Server) untuk memantau perubahan secara langsung:

   `npm run dev`

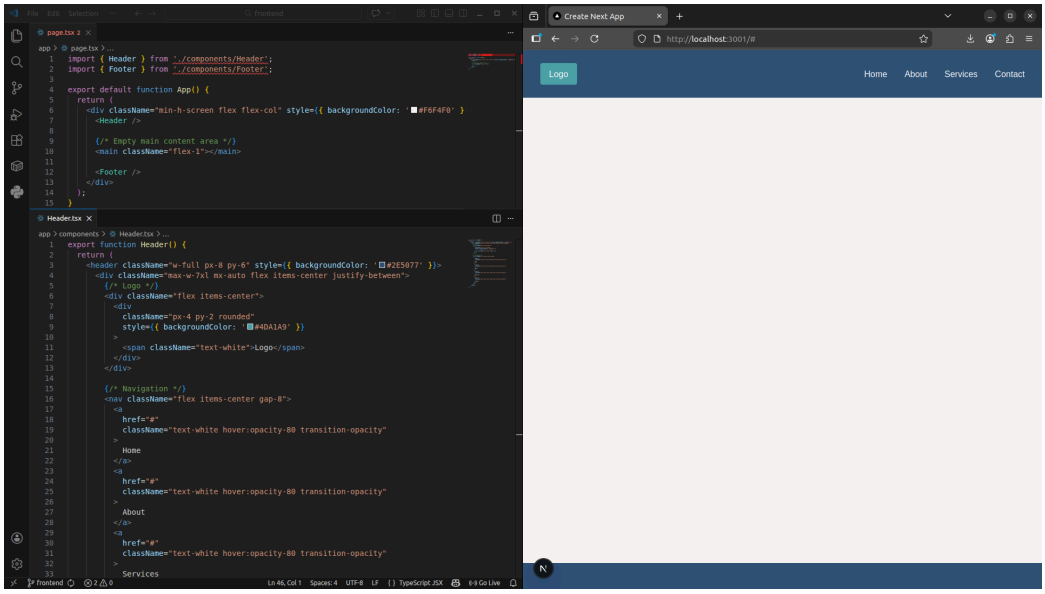


7. Buka *browser* dan akses tautan http://localhost:3000 untuk melihat pratinjau *website.*



B. Modifikasi Bagian Atas/Header Homepage

1. Buka folder frontend menggunakan *text editor*. Cari dan buka berkas utama untuk *Homepage* (terletak di `app/page.tsx`).
2. Lakukan perubahan pada kode antarmuka di bagian *header.* Pantau perubahannya secara *live* di browser.



3. Setelah perubahan disimpan, kembali ke terminal untuk mendaftarkan perubahan tersebut ke dalam Git:

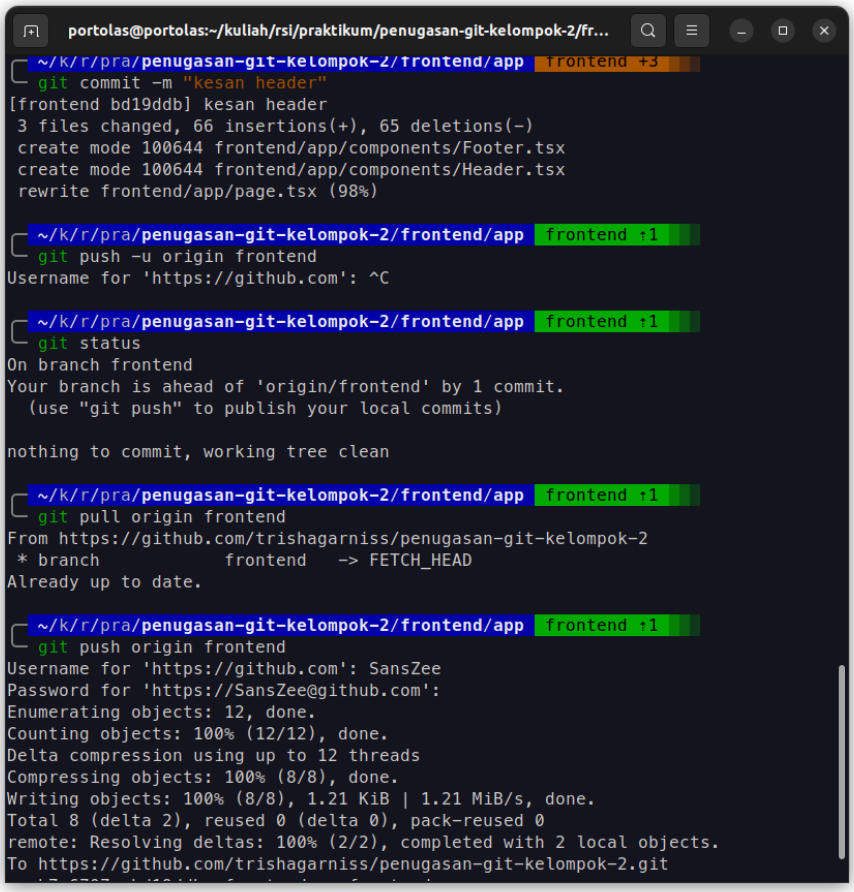
   `git add .`

4. Simpan perubahan dengan membuat *commit:*

   `git commit -m "MASUKKAN_PESAN_PERUBAHAN_DISINI"`

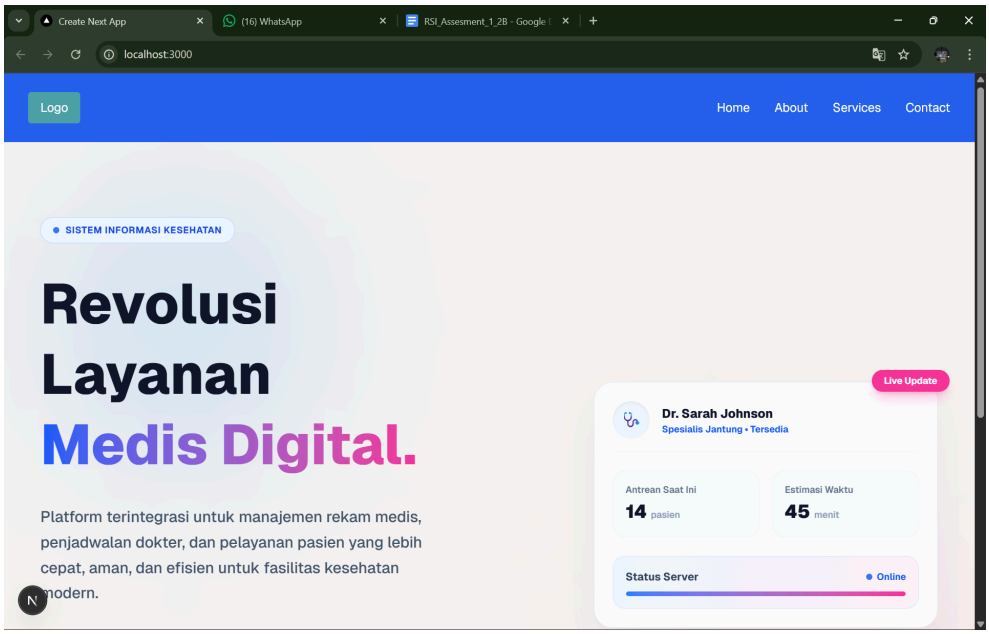
5. Unggah (*push*) kode yang telah diperbarui ke repository GitHub pada *branch* frontend:

   `git push origin frontend`



C. Modifikasi Bagian Tengah/Body Homepage

1. Buka folder frontend menggunakan *text editor*. Cari dan buka berkas utama untuk *Homepage* (terletak di `app/page.tsx`).
2. Lakukan perubahan pada kode antarmuka di bagian *header.* Pantau perubahannya secara *live* di browser.



3. Setelah perubahan disimpan, kembali ke terminal untuk mendaftarkan perubahan tersebut ke dalam Git:

   ```
   git add .
   ```

4. Simpan perubahan dengan membuat *commit:*

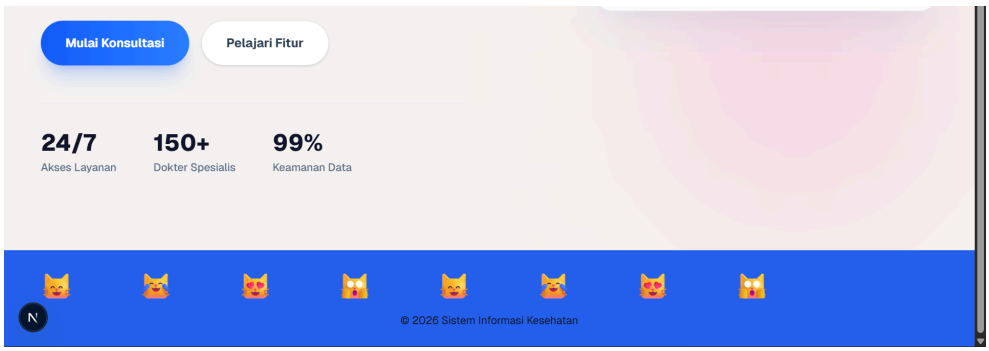
   ```
   git commit -m "MASUKKAN_PESAN_PERUBAHAN_DISINI"
   ```

5. Unggah (*push*) kode yang telah diperbarui ke repository GitHub pada *branch* frontend:

   ```
   git push origin frontend
   ```

D. Modifikasi Bagian Bawah/Footer Homepage

1. Buka folder frontend menggunakan *text editor*. Cari dan buka berkas utama untuk *Homepage* (terletak di `app/page.tsx`).
2. Lakukan perubahan pada kode antarmuka di bagian *footer*. Pantau perubahannya secara *live* di browser.



3. Setelah perubahan disimpan, kembali ke terminal untuk mendaftarkan perubahan tersebut ke dalam Git:

   ```
   git add .
   ```

4. Simpan perubahan dengan membuat *commit:*

   ```
   git commit -m "MASUKKAN_PESAN_PERUBAHAN_DISINI"
   ```

5. Unggah (*push*) kode yang telah diperbarui ke repository GitHub pada *branch* frontend:

   ```
   git push origin frontend
   ```

KESIMPULAN

Berdasarkan pengerjaan pengaturan repository Git untuk Kelompok 2, dapat disimpulkan beberapa poin penting dalam alur kerja kolaborasi pengembangan perangkat lunak:

1. **Isolasi Lingkungan Kerja (Branching)**

   Penggunaan *branch* yang terpisah (backend dan frontend) terbukti efektif untuk mencegah konflik kode antar anggota tim. Setiap *developer* dapat fokus pada *environment* masing-masing tanpa terganggu oleh *file* dari *role* lain.

2. **Manajemen Dependensi**

   File instalasi lokal yang berukuran besar dan spesifik terhadap terhadap sistem operasi (seperti `node_modules` untuk Node.js dan `.venv/__pycache__` untuk Python) tidak boleh dimasukkan ke dalam version control. Penggunaan file `.gitignore` mutlak diperlukan untuk menjaga repositori tetap ringan dan bersih.

3. **Sinkronisasi Konfigurasi**

   Meskipun *file* hasil instalasi diabaikan, *file* konfigurasi daftar dependensi (seperti `package.json` di *frontend* dan `pyproject.toml` serta `poetry.lock` di *backend*) wajib di-*push*. Hal ini memastikan setiap anggota tim yang melakukan *pull* dapat mereplikasi lingkungan kerja yang 100% identik di perangkat mereka masing-masing.